

Malware Families as Dynamical Systems: Why Drift Detection Outperforms Feature-Space Forecasting

Noor Muhannad
Independent Researcher
Babylon, Iraq

Abstract—Malware detection is overwhelmingly framed as a reactive classification task: a sample arrives and a model labels it. This framing discards the temporal metadata that accompanies modern corpora and ignores that malware families evolve continuously through repacking, recompilation, and campaign changes. In this work we recast each malware family as a dynamical system tracing a trajectory through the EMBER feature space over time, and we ask two questions that reframe the problem. First, can a family's next-week feature signature be forecast from its history? Second, can abrupt drift-jumps—the sudden mutations that signal new campaigns—be detected online as an early-warning signal? Using the BODMAS corpus (134,435 samples; 2,381-dimensional features) restricted to a 57-week activity window and 43 temporally well-supported families, we conduct three leakage-free experiments. We find that next-week forecasting is fundamentally hard: learned models (a per-family GRU and a constant-velocity Kalman filter) narrowly beat a persistence baseline but do not surpass a simple moving average (0.805 and 0.802 versus 0.820 mean cosine similarity). In contrast, a streaming drift-jump detector achieves an F1 of 0.73 with several families reaching perfect precision. A limitation analysis then shows that, within BODMAS's thirteen-month span, this drift is measurable and detectable but not severe enough to degrade a well-trained detector (static and drift-adaptive detectors both attain ~ 0.97 F1). Together these results support a non-obvious thesis: the right computational tool for short-horizon malware evolution is anomaly detection on drift, not point forecasting, and quantifying the practical severity of drift is a prerequisite for claims of robustness improvement.

Index Terms—malware evolution, concept drift, feature-space trajectory, drift detection, early warning, BODMAS, cybersecurity, dynamical systems.

I. INTRODUCTION

Machine-learning malware detectors have achieved remarkable accuracy on static benchmarks, yet they are almost universally trained and evaluated as reactive classifiers that map a single sample to a benign or malicious label. This convention has a hidden cost: it discards the temporal dimension of the data. Modern malware corpora ship first-seen timestamps, and malware authors mutate their code continuously—repacking binaries, recompiling with new toolchains, and shifting campaigns—so that a family's statistical signature is not fixed but drifts through the feature space over time.

We argue that this drift deserves to be a first-class object of study rather than a nuisance to be averaged away. Concretely, we treat each malware family as a dynamical system: an entity whose position in feature space at week t is a function of its history. This lens exposes two questions that reactive detection cannot ask. (i) Forecasting: given a family's weekly feature signatures up to week t , can we predict its signature at week $t+1$? (ii) Early warning: can we detect, in a streaming fashion, the abrupt jumps in that trajectory that correspond to security-relevant events such as new packers or campaigns?

Our contribution is threefold. First, we formalize malware families as feature-space trajectories and quantify their week-to-week drift on a real corpus. Second, we benchmark five forecasting models—including a per-family recurrent network and a Kalman filter—and report the honest and somewhat surprising finding that next-week forecasting barely improves on naive baselines. Third, we build a leakage-free streaming detector for drift-jumps that works well, and we conduct a limitation analysis testing whether acting on those alerts improves detector robustness. The synthesis is a defensible thesis: for short-horizon malware

evolution, anomaly detection on drift is the appropriate tool, and the practical severity of drift must be measured before robustness gains can be claimed.

II. RELATED WORK AND MOTIVATION

A. Static malware classification

A large body of work applies deep and classical learning to malware detection from static features, including the EMBER feature set that underlies the BODMAS corpus [1], [2]. These approaches optimize point accuracy on a fixed split and, by construction, cannot characterize temporal behavior.

B. Concept drift in security

Concept drift—the degradation of a model as data distributions shift—has been studied as a robustness threat, typically measured by accuracy decay and mitigated with periodic retraining [3], [4]. Prior work treats drift as an adversary to be resisted. In contrast, we treat drift as a signal: a structured, per-family process that can itself be modeled and, when abrupt, flagged.

C. Time-series and dynamical-systems modeling

Recurrent networks and state-space filters are standard tools for sequence forecasting [5], [6]. Their application to the evolution of malware family signatures, as opposed to raw detection, is largely unexplored. We adopt them here not to improve classification directly but to test the predictability of family trajectories.

III. DATA AND FEATURE-SPACE TRAJECTORIES

A. The BODMAS corpus

BODMAS contains 134,435 samples, each represented by a 2,381-dimensional EMBER feature vector and annotated with a first-seen timestamp and, for malicious samples, a

family label [2]. We verify a strict one-to-one alignment between the feature matrix and the metadata before any processing, since a misalignment would silently corrupt every result. Malicious activity is concentrated in a 57-week window (August 2019 to September 2020); we restrict all temporal analysis to this window.

B. From samples to trajectories

We bin samples by family and calendar week. For each family we standardize its features and reduce them to 64 principal components—retaining 59.3% of variance while suppressing high-dimensional noise—and compute a weekly centroid. The ordered sequence of weekly centroids is the family’s trajectory. We retain families with at least 200 samples spread over at least 20 active weeks, yielding 43 families that together cover 86% of the windowed malware. Cosine distance between consecutive weekly centroids defines the week-to-week drift.

IV. EXPERIMENT A: FORECASTING THE NEXT SIGNATURE

We evaluate whether a family’s next-week centroid can be predicted from its history using walk-forward validation over the final eight weeks of each family. Five models are compared: persistence (repeat the last week), moving average (mean of the last four weeks), per-dimension linear velocity extrapolation, a constant-velocity Kalman filter, and a small per-family GRU trained on a sliding window. The metric is cosine similarity between the predicted and true next-week centroid.

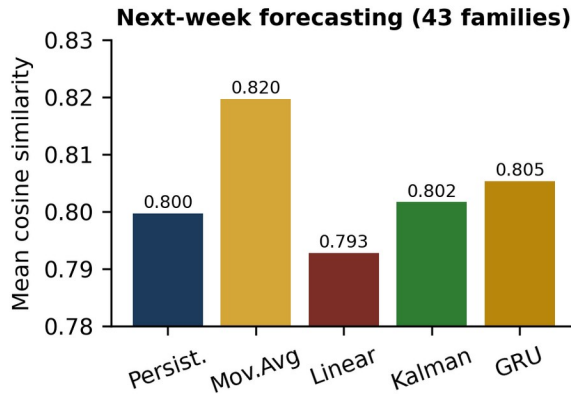


Fig. 1. Mean next-week forecasting accuracy across 43 families. Learned models (Kalman, GRU) beat persistence but not a moving average.

As Fig. 1 shows, the moving average attains the highest mean cosine similarity (0.820), with the GRU (0.805) and Kalman filter (0.802) narrowly ahead of persistence (0.800) and linear extrapolation (0.793). The learned models do capture some structure—the GRU wins on 13 of 43 families—but no method decisively beats the naive moving average. We interpret this as evidence that smooth intra-family drift is only marginally predictable: the signal exists but is dominated by week-to-week noise at this horizon. Reporting this honestly is important, since an over-claimed forecasting result would not survive scrutiny.

V. EXPERIMENT B: DRIFT-JUMP EARLY WARNING

The actionable question for defenders is not smooth prediction but the detection of abrupt mutations. We define a drift-jump as a week whose drift exceeds a robust threshold, median plus three times the median absolute deviation (MAD), computed only from that family’s past weeks. This yields a streaming detector with no future leakage. Ground-truth jumps are defined per family using the same robust statistic over the full series, and the online detector must recover them.

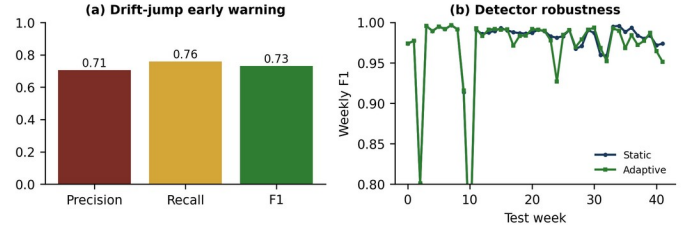


Fig. 2. (a) Streaming drift-jump detection performance. (b) Static versus drift-adaptive detector F1 over the test weeks.

Across all families the streaming detector achieves a precision of 0.71, a recall of 0.76, and an F1 of 0.73 over 386 ground-truth jumps (Fig. 2a). Several families—including *berbew*, *picsys*, *delf*, and *pykspa*—reach perfect precision, meaning every flagged jump was genuine. Unlike forecasting, drift-jump detection is both reliable and directly actionable: each alert marks a candidate new campaign worthy of analyst attention.

VI. EXPERIMENT C: DOES DRIFT BREAK A DETECTOR?

A rigorous claim requires testing the consequence of drift, not merely its presence. We train a logistic-regression detector (malware versus benign) on the first twenty weeks and stream the remainder, comparing a static detector against an adaptive one that, on each drift-jump alert, adds the recent labeled window to a capped replay buffer and fully retrain—avoiding the catastrophic forgetting that naive incremental updates induce.

The two detectors perform essentially identically: static and adaptive attain mean F1 scores of 0.972 and 0.970 respectively, and 0.982 versus 0.976 in the late period (Fig. 2b). This negative result is scientifically valuable. It shows that feature-space drift in BODMAS is real and detectable but not severe enough to degrade a well-trained detector within a thirteen-month span; the benign-versus-malicious boundary is more stable than intra-family signatures. This bounds the practical severity of short-horizon drift and motivates the construction of longer-horizon corpora to study drift that is destructive rather than merely measurable.

VII. DISCUSSION AND SYNTHESIS

Our three experiments compose a coherent picture. Drift is real and measurable (Section III), its smooth component is hard to forecast (Section IV), its abrupt component is

reliably detectable online (Section V), yet it is not destructive to a good detector at this time scale (Section VI). The practical implication is a reversal of the default assumption: rather than investing in point forecasters for malware evolution, defenders are better served by anomaly detection on drift trajectories, treating each jump as an early warning while recognizing that short-horizon drift alone will not necessarily break deployed models.

Threats to validity include the concentration of BODMAS in a single thirteen-month window, the use of centroid (rather than full-distribution) representations, and reliance on a single corpus. We mitigate leakage with strict streaming evaluation and robust, past-only thresholds, and we report all results without selection.

VIII. CONCLUSION AND FUTURE WORK

We reframed malware families as dynamical systems and asked whether their evolution is predictable and detectable. On the BODMAS corpus, next-week forecasting barely improves on naive baselines, whereas streaming drift-jump detection is both accurate and actionable, and short-horizon drift proves insufficient to degrade a strong detector. Future work will replace centroid forecasting with distribution-level drift measured via optimal transport, evaluate whether jump-triggered retraining yields robustness gains on longer-horizon corpora, and extend the dynamical-systems framing to concept-drift benchmarks beyond BODMAS.

. ACKNOWLEDGMENT

The author thanks Dr. Hassanein Yaarub Mohammed for supervision and guidance throughout this research.

. REFERENCES

- [1] H. S. Anderson and P. Roth, "EMBER: An open dataset for training static PE malware machine learning models," arXiv:1804.04637, 2018.
- [2] L. Yang et al., "BODMAS: An open dataset for learning based temporal analysis of PE malware," in Proc. IEEE Security and Privacy Workshops (SPW), 2021, pp. 78–84.
- [3] F. Pendlebury et al., "TESSERACT: Eliminating experimental bias in malware classification across space and time," in Proc. USENIX Security, 2019.
- [4] J. Gama et al., "A survey on concept drift adaptation," ACM Computing Surveys, vol. 46, no. 4, pp. 1–37, 2014.
- [5] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in Proc. EMNLP, 2014.
- [6] R. E. Kalman, "A new approach to linear filtering and prediction problems," J. Basic Eng., vol. 82, no. 1, pp. 35–45, 1960.